

Exercice 1 (Pixel War - 10 pts)

Nous souhaitons développer une application réseau qui permet de réaliser une fresque collective en pixels. Cette dernière fait 10000 par 10000 pixels et est stockée dans un fichier binaire, ligne par ligne, chaque pixel correspondant à 3 unsigned char (nous supposons l'existence de la structure pixel_t).

L'application repose sur un serveur UDP et des clients. Les clients peuvent effectuer deux types de requête : la récupération d'une portion de la fresque de 20 par 20 pixels et la modification d'un pixel.

1°) Expliquez quelles données sont échangées dans l'application et donnez les structures C correspondantes.

Solution : (1 pt)

- *Demande d'une portion : type (unsigned char) + coordonnées (2 unsigned short) + adresse de réponse (inclue dans la requête UDP)*
- *Envoi de portion : tableau de 20 par 20 pixels (20x20x3 unsigned char)*
- *Demande de modification : type (unsigned char) + coordonnées (2 unsigned short) + valeur du pixel (3 unsigned char)*

```
typedef struct {
    unsigned char type;
    unsigned short x, y;
    pixel_t pixel;
} requete_t;
typedef struct {
    pixel_t portion[20][20];
} reponse_t;
```

2°) Écrivez la fonction ouvrir qui permet d'ouvrir le fichier contenant la fresque. Le nom du fichier est passé en paramètre. Si le fichier n'existe pas, il est créé (une fresque avec des pixels noirs (0,0,0) est alors créée). La fonction retourne le descripteur de fichier. La vérification de l'existence du fichier doit être effectuée à l'aide de la fonction open et de la gestion d'erreur.

Solution : (3 pts)

```
int ouvrir(char *nomFichier) {
    int fd = open(nomFichier, O_RDWR);
    pixel_t ligne[10000];

    if(fd == -1) {
        if(errno == ENOENT) {
            fd = open(nomFichier, O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
            memset(ligne, 0, sizeof(ligne));
            for(i = 0; i < 10000; i++)
                write(fd, ligne, sizeof(ligne));
        }
    }
    return fd;
}
```

3°) Nous supposons l'existence de la fonction setpixel qui prend en paramètres le descripteur de fichier, les coordonnées (x,y) du pixel et sa couleur (de type pixel_t). Elle modifie le pixel correspondant. Sans donner le code, expliquez ce que fait cette fonction en précisant les appels systèmes nécessaires.

Solution : (1 pt) A l'aide du descripteur de fichier, un appel à fseek est effectué pour se placer à la bonne position dans le fichier. Ensuite, un appel à write est réalisé pour écrire la nouvelle valeur du pixel.

4°) Donnez le code du serveur permettant d'initialiser la socket. Le numéro de port est fixé par une constante PORT.

Solution : (2 pts)

```
int sockfd;
struct sockaddr_in adresseServeur;

sockfd = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

memset(&adresseServeur, 0, sizeof(struct sockaddr_in));
adresseServeur.sin_family = AF_INET;
adresseServeur.sin_port = htons(PORT);
adresseServeur.sin_addr.s_addr = htonl(INADDR_ANY);

bind(sockfd, (struct sockaddr*)&adresseServeur, sizeof(struct sockaddr_in));
```

5°) À votre avis, est-il nécessaire de mettre en place une solution parallèle pour répondre aux requêtes des clients ? Si oui, expliquez.

Solution : (1 pt) *Le traitement est assez léger : le traitement en parallèle n'est donc pas nécessaire.*

6°) Donnez la portion de code permettant au serveur de recevoir les requêtes et d'y répondre.

Solution : (2 pts)

```
struct sockaddr_in adresseClient;
socklen_t longueurAdresse = sizeof(struct sockaddr_in);
requete_t requete;
reponse_t reponse;

while(stop == 0) {
    recvfrom(sockfd, &requete, sizeof(requete_t), 0,
              (struct sockaddr*)&adresseClient, &longueurAdresse);

    switch(requete.type) {
        case 1:
            setpixel(fd, requete.x, requete.y, requete.pixel);
            break;
        case 2:
            for(i = 0; i < 20; i++) {
                lseek(fd, y*10000*sizeof(pixel_t)+x, SEEK_SET);
                read(reponse.portion[i], sizeof(pixel_t)*20);
                sendto(sockfd, &reponse, sizeof(reponse_t), 0,
                       (struct sockaddr*)&adresseClient, longueurAdresse);
            }
            break;
    }
}
```

7°) Nous supposons qu'un client donné ne peut modifier un pixel que toutes les 5 secondes minimum. Proposez une solution permettant de résoudre cette nouvelle contrainte.

Solution : (1 pts) *Le plus simple est de conserver la liste des adresses des clients avec un timestamp. Quand on reçoit une requête, on vérifie simplement que la date actuelle et le timestamp ont bien une différence de 5s. Si c'est le cas, on applique la modification et on met à jour la liste. Sinon, on ignore la requête.*

Exercice 2 (File de messages partagée (les IPC) - 10 pts)

Nous souhaitons mettre en place une architecture distribuée de communication entre des clients et un contrôleur à l'aide d'une file de messages partagée. Les clients peuvent déposer des messages qui sont lus par les autres clients ou le contrôleur. Seul le contrôleur peut supprimer les messages de la file.

Chaque client est caractérisé par son statut : un PID, la date de connexion et la date de dernière émission dans la file. Le nombre maximum de clients est fixé à l'aide d'une constante `MAX_CLT = 10`. La file de

messages, représentée par un tableau, contient un nombre maximal d'entrées fixé par une constante `MAX_ENT = 20`. Chaque entrée est caractérisée par le PID de l'émetteur et un message (une chaîne de caractères de taille fixe spécifiée par la constante `MAX_MSG = 256`). La file de messages et les statuts des clients sont stockés dans un segment de mémoire partagée. La gestion de la concurrence est réalisée à l'aide de sémaphores.

Pour accéder à la file, chaque client doit être connecté : il doit d'abord trouver une place disponible parmi les statuts et spécifier son statut.

Pour rappel, une file circulaire de taille `T` représentée à l'aide d'un tableau utilise deux entiers : les indices de début (`D`) et de fin (`F`). Lorsque `F=D`, la file est vide. Lorsque `(D+1) % T = F`, la file est pleine (en réalité, la file ne peut contenir que `T-1` éléments). Le prochain élément à lire est situé dans la case dont l'indice est `F` (ensuite `F = (F+1) % T`). Un nouvel élément doit être placé dans la case d'indice `D` (ensuite `D = (D+1) % T`).

1°) Expliquez pourquoi une file de messages IPC ne serait pas appropriée dans le cadre de cette application.

Solution : (1 point) *La lecture dans une file supprime les messages. Du coup, si un client souhaite lire les messages des autres clients, ceux-ci sont supprimés et il faut ensuite les remplacer. Cela serait trop coûteux en termes d'appels systèmes et changerait l'ordre des messages.*

2°) Donnez le contenu du segment de mémoire partagée, en précisant le type des données. Spécifiez les structures nécessaires (statut et message).

Solution : (2 points) *Le statut est caractérisé par la structure suivante :*

```
typedef struct {
    pid_t pid;          /* PID du processus */
    time_t connexion;   /* Date de connexion */
    time_t emission;    /* Dernière émission */
} statut_t;
```

Un message de la file est caractérisé par la structure suivante :

```
typedef struct {
    pid_t pid;          /* PID du processus */
    char message[MAX_MSG]; /* Le message */
} message_t;
```

Le segment est donc constitué des éléments suivants :

- Les statuts : `statut_t [MAX_CLT]`
- La file de messages :
 - deux entiers (`F` et `D`)
 - les entrées : `message_t [MAX_ENT+1]`

Le tableau de sémaphores et le segment de mémoire partagée sont créés et initialisés par le contrôleur. Nous supposons que la limitation du nombre de clients est gérée à l'aide d'un sémaphore. Lorsqu'un client démarre, si le nombre maximal de clients est atteint, il doit s'arrêter et ne pas rester bloqué. Dès qu'un client se connecte, il spécifie son statut dans le segment de mémoire partagée dans un emplacement libre. Il peut ensuite ajouter des messages dans la file ou les consulter.

3°) Donnez les sémaphores nécessaires pour la phase de connexion, ainsi que leur initialisation.

Solution : (1 point) *Nous avons besoin d'un sémaphore pour le nombre de clients (`S_1`) initialisé à `MAX_CLT` et d'un sémaphore pour permettre l'exclusion mutuelle sur les statuts (`S_2`) initialisé à 1.*

4°) Donnez toutes les étapes nécessaires du démarrage d'un client jusqu'à la spécification de son statut dans le segment de mémoire partagée. Vous préciserez tous les appels systèmes ainsi que les sémaphores nécessaires (en indiquant leur valeur d'initialisation).

Solution : (3 points) Le client doit récupérer le tableau de sémaphores (`semget`). Il doit ensuite faire une opération non bloquante pour décrémenter le sémaphore `S_1` (`semop`). Si l'opération échoue, le client s'arrête. Sinon, il récupère le segment de mémoire partagée (`shmget`) et l'attache à sa mémoire (`shmat`). Il doit demander l'exclusion mutuelle sur les statuts avec le sémaphore `S_2` (`semop`) puis rechercher dans le tableau un statut libre (`PID=0`, par exemple). Il spécifie ensuite son statut puis libère l'exclusion mutuelle sur le sémaphore `S_2` (`semop`).

Pour éviter que des messages ne soient perdus, il faut empêcher les clients d'envoyer un message si la file est pleine. Par contre, les autres clients et le contrôleur doit pouvoir continuer à lire les messages.

5°) Proposez une solution à l'aide de sémaphores, tout en limitant le nombre d'appels systèmes et en évitant l'attente active. Décrivez précisément le fonctionnement de votre solution (ajout et lecture des messages).

Solution : (3 points) Une solution consiste à utiliser un sémaphore `S_3` initialisé à `MAX_ENT` et un second `S_4` initialisé à 1. Lorsqu'un client désire écrire un message, il doit décrémenter de 1 les deux sémaphores en réalisant les deux opérations en un seul appel atomique. Si la file est pleine, il sera bloqué avec le sémaphore `S_3`. Si le contrôleur ou un autre client est en train de lire ou d'écrire dans la file, il sera également bloqué avec le sémaphore `S_4`. Ainsi, il ne peut écrire que si les deux opérations sont possibles. Une fois le message ajouté, il peut incrémenter `S_4` de 1 pour libérer l'exclusion mutuelle. Le contrôleur ou un client qui désire lire les messages doivent simplement décrémenter le sémaphore `S_4`. Si un client est en train de lire ou d'écrire, ils seront bloqués. Pour chaque message supprimé, le contrôleur incrémente `S_3`. Une fois que l'accès à la file n'est plus nécessaire, on incrémente `S_4` pour libérer l'exclusion mutuelle.